插件化 Agent：一个目录定义一个会自己跑的 Agent

开篇：为什么我们敢叫"Agent OS"

先回答一个这门课自始至终悬着的问题——**凭什么叫 OS？** 这个词不是修辞，得对得起。

看操作系统怎么成立的：一个能跑单个程序的东西不叫 OS，它只是那个程序。**OS 的定义是"一套内核之上，能装、能跑任意多个程序，彼此隔离、不改内核、免重编译"**——是 Word、Chrome、Python 一起跑在同一个 Windows 上，Windows 自己一行不改。让它成为 OS 的，从来不是内核有多强，而是那套**"定义一个程序、把它装上去、让它跑起来"**的标准机制（可执行文件格式 + 加载器 + 调度）。

套到我们身上：**Agent OS = 一套底座之上，能装、能跑任意多个 Agent。** 到 28 节为止，我们把**底座**（内核）造齐了、也跑稳了——Provider、ReAct、内置 Tool、Memory、Sandbox、定时、Web，这些是**系统的基础能力**，所有 Agent 共享。但内核不等于 OS：此刻想跑一个业务 Agent，还得把指令硬写进配置、手工拼装。**缺的正是那套"定义一个 Agent、把它装上去、让它跑起来"的标准机制**——就像一个只有内核、没有可执行文件格式和加载器的系统，称不上 OS。

这一节补的就是这块，所以它是 OryxOS 能被叫作 Agent OS 的核心一节。 做完它，"定义一个 Agent"退化成**往 `.oryxos/agents/` 丢一个目录**——任意多个、互不干扰、不动底座、到点自己跑。从这一节起，OryxOS 才从"一个能跑 Agent 的框架"真正变成"一台能跑 N 个 Agent 的 OS"。

一、本节目标

1.1 两层：底座（系统基础能力） + Agent（一个自足的目录）

类比操作系统看得最清楚：

层	OS 类比	OryxOS	是什么
底座	内核 + 系统调用	Provider、ReAct、内置 Tool（`read_file` / `shell` / `http_get` / `notify` / `save_memory` …）、Memory、Sandbox、定时、Web（16~28 节）	**系统基础能力**，所有 Agent 共享
Agent	一个可执行程	一个 **`.oryxos/agents/<name>/` 目录**	一个自足的业务 Agent：

层	OS 类比	OryxOS	是什么
	序（一个目录/包）		自带身份/配置、指令、脚本、参考

终点（可验证的终态）：往 `.oryxos/agents/` 丢一个 Agent 目录 → 它出现在 Agent 列表里 → 到它自己声明的时间点，自动跑完"想 → 调系统能力 → 答 → 推送"、审计留账 → 改一下它目录里的指令，下一轮触发即时生效 → 全程不写一行 Java、不动一行底座。

1.2 借 Anthropic Agent Skills 的"形态"，但定义的是 Agent

这套模型的**形态**借鉴 **Anthropic Agent Skills**——但要说清一件容易混的事：

> Anthropic 把"一个装着 `SKILL.md` + 脚本 + 参考的目录"叫一个 **Skill**（Claude 这个大 Agent 的一项可加载能力）。**我们借的是这个目录的形态，不是这个命名**——在 OryxOS 里，**这样一个目录定义的是一个 Agent**（一个会自己跑的业务程序），不是一项挂在别人身上的能力。

所以关键区别一句话记住：**在 OryxOS，一个目录 = 一个 Agent；"skill"只是这个 Agent 目录里的一个组成部分**（可选的子指令 `.md`），不是顶层单位。我们**不做**"跨 Agent 的共享能力库"——每个 Agent 独立自足，只调用底座的系统基础能力。

它从 Anthropic 形态里真正拿来用的，是**渐进式披露（progressive disclosure）**——一个 Agent 目录里的东西**按需分阶段进上下文**，不一次性全塞：

- **指令正文**：Agent 的任务说明，被触发时进 system prompt（它就是这个 Agent 的"人格 + 干什么"）。
- **参考 / 子指令（`REFERENCE.md`、`skills/*.md`）**：用到才读——Agent 按正文指引，用底座的 `read_file` 把它读进来。
- **脚本（`scripts/*.py`）**：用到才跑——用底座的 `shell` / `python` 执行，**产出进上下文、代码不进**，确定性操作不烧 token。

> **和上一版课件的关系**：早先几版在"一份 Skill = 一个 Agent"和"Skill = 可复用能力"之间摇摆过。这一版定死：**一个目录 = 一个 Agent**，skill 收进 Agent 内部当组成。不再有跨 Agent 的 `use_skill` /能力库/全局索引——那套是"多 Agent 共享能力"的方案，我们不走。

1.3 一个 Agent 目录长什么样

全节用一个真实例子贯穿——**每日订单对账 Agent**：每天早上核对交易库与清算库昨天订单的条数和金额是否一致，有差异就按规范生成分级报告推到运维群。它一个目录就把"正文常驻 / 子指令按需 / 脚本确定性 / 参考兜底"四样都用上，是这套模型最好的活教材。

```
.oryxos/agents/daily-reconcile/      # 一个目录 = 一个 Agent
├── AGENT.md                         # 主文件：frontmatter(这个 Agent 的
profile) + 正文(任务指令)
├── REFERENCE.md                     # 可选：参考资料（字段字典/已知可接受差异），拿
不准才读（read_file）
├── skills/                          # 可选：子指令（Agent 内部的一部分），用到才读
│   └── report-format.md             #   差异报告规范 + P0/P1/P2 分级
└── scripts/                         # 可选：脚本，用到才跑（shell/python），代码
不进上下文
    └── reconcile.py                 #   连两库、逐单比对、输出差异 JSON
```

`AGENT.md` （frontmatter = 这个 Agent 的 profile；正文 = 任务指令）：

```
---
name: daily-reconcile                # 唯一标识 = 目录名 = 这个 Agent 的名字
description: 每天核对交易库与清算库当日订单的条数与金额是否一致；有差异就按规范生成分级报
告并推送
identity:
  agent_name: 对账小欧
  prompt: 你是一个严谨的对账助手，只根据脚本给出的确定性数据下结论，绝不臆测数字。
provider:                            # 这个 Agent 自己的运行配置（就是它的
profile）
  name: deepseek
  model: deepseek-chat
  temperature: 0.2
tools: [shell, read_file, notify, save_memory]   # 它要用的系统基础能力（最小权
限，20 节）
notify_channels:
  - type: webhook
    url: ${OPS_WEBHOOK_URL}
schedules:                           # 它什么时候自己跑（定时属于 Agent）
  - {id: reconcile-morning, cron: "0 0 9 * * *", zone: Asia/Shanghai,
     message: 到点了，核对昨天的订单对账。}
---

你是每日订单对账助手。被触发时，严格按顺序做，不要跳步：
1. **拿数据（交给脚本）**：运行 `python scripts/reconcile.py`，它返回一段 JSON：
   `{date, orders_count, settle_count, orders_amount, settle_amount, diffs:
[{order_id,kind,detail}]}`。只依据它下结论。
2. **判断**：`diffs` 为空且条数、金额都相等 → 调 notify 发「✅ 对账通过」并结束；否则
进第 3 步。
3. **写报告（规范较长，用到才读）**：读 `skills/report-format.md` 按它的结构和
P0/P1/P2 分级组织报告；
   某条差异的字段含义或是否属于已知可接受差异拿不准，读 `REFERENCE.md` 对照后再定级。
```

```
4. **推送 + 留痕**：调 notify 推送报告；调 save_memory 记一笔「{date} 差异 {N} 笔，最高 {P?}，已通知」。
```

- **frontmatter = 这个 Agent 自己的 profile**：身份、provider/model、要用的系统能力、往哪推、什么时候跑——**一个 Agent 自带一切，不再另写一份 Profile YAML**（`.oryxos/profiles/` 取消）。
- **正文 = 给 LLM 的任务指令**；`skills/`、`REFERENCE.md`、`scripts/` 是这个 Agent 按需读/跑的自有资源。
- 存放位置固定 `.oryxos/agents/<name>/`，**一个目录一个 Agent**。业务方新增一个 Agent，从头到尾就写这一个目录。

1.4 这个 Agent 的另外三个文件：各自"用到才进上下文"

正文只做**编排**；真正的报告规范、字段参考、抓数逻辑分在另外三个文件里，各自按需加载——这正是一个 Agent 内部的渐进式披露：

文件	角色	何时进上下文	靠哪个底座能力
`AGENT.md` 正文	任务编排	触发即**常驻** system prompt	`ContextLoader` 注入
`scripts/reconcile.py`	确定性抓数 / 比对	跑它时，**只有输出 JSON 进**、代码不进	`shell` / `python`
`skills/report-format.md`	报告规范 + 分级	**出现差异那一步**才读进来	`read_file`
`REFERENCE.md`	字段字典 / 已知差异兜底	某条差异**定级拿不准**才读	`read_file`

`scripts/reconcile.py`（确定性抓数、纯标准库、无 key；进上下文的是它吐的 JSON，代码不进）：

```python
import csv, json, os, sys
from datetime import date, timedelta

def load(path):
    rows = {}
    with open(path, newline="", encoding="utf-8") as f:
        for r in csv.DictReader(f):
            rows[r["order_id"]] = round(float(r["amount"]), 2)
    return rows

orders_path = os.environ.get("RECON_ORDERS_CSV")    # 交易库昨日导出
settle_path = os.environ.get("RECON_SETTLE_CSV")    # 清算库昨日导出
```

```
if not orders_path or not settle_path:
    json.dump({"error": "请设置 RECON_ORDERS_CSV 与 RECON_SETTLE_CSV 指向昨日两
库导出"},
              sys.stdout, ensure_ascii=False); sys.exit(0)

orders, settle, diffs = load(orders_path), load(settle_path), []
for oid in orders.keys() - settle.keys():
    diffs.append({"order_id": oid, "kind": "missing_in_settle", "detail":
"交易库有、清算库无 (%.2f) " % orders[oid]})
for oid in settle.keys() - orders.keys():
    diffs.append({"order_id": oid, "kind": "missing_in_orders", "detail":
"清算库有、交易库无 (%.2f) " % settle[oid]})
for oid in orders.keys() & settle.keys():
    if orders[oid] != settle[oid]:
        diffs.append({"order_id": oid, "kind": "amount_mismatch", "detail":
"金额不符: 交易 %.2f vs 清算 %.2f" % (orders[oid], settle[oid])})

json.dump({
    "date": (date.today() - timedelta(days=1)).isoformat(),
    "orders_count": len(orders), "settle_count": len(settle),
    "orders_amount": round(sum(orders.values()), 2), "settle_amount":
round(sum(settle.values()), 2),
    "diffs": sorted(diffs, key=lambda d: d["order_id"]),
}, sys.stdout, ensure_ascii=False)
```

`skills/report-format.md`（子指令：报告结构 + 分级规则，较长，出现差异那步才 `read_file`）：

```
# 对账差异报告规范
## 分级规则（取命中的最严重一条作为整份报告级别；定级前先剔除 REFERENCE.md 里的已知可接
受差异/测试单号）
- P0（立即处理）：amount_mismatch 金额差合计 > 10000 元，或任一单笔差 > 5000 元 —— 
可能资损，立即升级。
- P1（当天处理）：出现 missing_in_settle（交易库有、清算库无）≥ 1 笔 —— 订单没进清算，
当天排查。
- P2（观察）：仅 missing_in_orders，或仅单笔 0.01 元尾差 —— 多为跨天入账/尾差，观察即
可。
## 报告结构
标题 `【对账 {级别}】{date} 差异 {N} 笔`；总览一行（条数/金额 两库对照）；按 kind 分组
各列前 10 条
`- {order_id} · {detail}`，超 10 条注明省略；结尾一行处置建议（P0 联系清算值班 / P1
当天排查 / P2 明日复核）。
报告只放结论，不粘脚本代码或原始 JSON。
```

`REFERENCE.md`（参考：字段字典 + 已知可接受差异，某条差异拿不准才 `read_file`）：

```
# 对账参考资料（拿不准某条差异时再读）
## 字段对照：orders(order_id 交易单号，amount 下单金额/元) ↔
settlements(order_id，amount 入账金额/元)，一一对应、正常应相等。
## 已知可接受差异（定级前剔除，不升 P0/P1）：
- 跨天入账：22:00 后订单清算次日入账 → 表现为 missing_in_orders，属正常。
- 测试单号：order_id 以 TEST- 开头是压测数据，忽略。
- 一分钱尾差：单笔差 = 0.01 元，历史四舍五入遗留，记 P2。
## 升级联系人：清算值班（P0）飞书群「清算-值班」；交易侧（P1 未入账）飞书群「交易-订单」。
```

四个文件合起来就是一个完整、能跑的 Agent。**本节开发时，spec-kit 按这份课件把 `daily-reconcile/` 作为示例交付物产出**（见"本节交付物·文件"）——课件是输入，代码由 spec-kit 生成。

二、围绕目标要做哪些

贯穿始终一条原则：**不重写底座，只给"Profile"添一个来源。** 底座（16~28 节）——`AgentService`、`ReActLoop`、`PromptBuilder`、`AgentScheduler`——都是吃 `Profile` 这个值对象的。所以我们一行底座都不动，只是把"从 Profile YAML 读 Profile"换成"**从 Agent 目录的 `AGENT.md` frontmatter 派生 Profile**"，走同一套注册和校验。

2.1 扫描：把每个 Agent 目录变成一个已注册的 Agent

系统启动时（以及 30 节通过 API 新增时）扫描 `.oryxos/agents/`，对每个目录做三步——**这三步就是"插件化"的机制本体**：

1. **解析**：`AgentLoader` 读目录里的 `AGENT.md`，拆出 frontmatter（配置）与正文（指令），记住 `scripts/`、`skills/`、`REFERENCE.md` 等资源所在。
2. **派生**：`deriveProfile(agentDir)` 把 frontmatter 映射成一个底座认识的 `Profile` 值对象（name / provider / tools / notify_channels / schedules 一一对应，正文和资源目录绑定到它）。
3. **注册**：把派生出的 Profile 塞进 `ProfileRegistry`（复用 16 节那套校验：provider 存在、tool 已注册），有 `schedules` 的再交给 `AgentScheduler` 注册定时。

为什么走"派生 Profile"、而不是另起一套？ 因为底座 16~28 节的一切都吃 `Profile`。派生成 Profile，就等于让 Agent 目录**零改动复用整台底座**。运行时它触发一次，跟 CLI / Web 人推走的是同一个 `AgentService.process`，ReAct/Tool/Provider 一个字不用改。

2.2 渐进式披露：一个 Agent 目录里的东西按需进上下文

Agent 的**指令正文**在被触发时进 system prompt（`ContextLoader` 供给，跟 Bootstrap 同层，无缓存、改完即时生效）。而它自带的**参考、子指令、脚本**不预先全塞进去，按正文指引**用底座的系统基础能力按需取用**：

- 读参考 / 子指令：正文说"报告规范见 `skills/report-format.md`" → 模型用底座的 **`read_file`** 把它读进上下文；
- 跑脚本：正文说"运行 `python scripts/reconcile.py`" → 模型用底座的 **`shell` / `python`** 跑，**脚本产出进上下文、代码不进**。

这里没有新工具、没有能力库、没有全局索引——渐进式披露完全靠"正文是判断、资源用底座既有的 `read_file` / `shell` 按需取"实现。一个 Agent 内部的资源加载，天然被限制在它自己的目录里（沙箱见 2.4）。

2.3 定时来自 Agent 自己

定时写在 `AGENT.md` 的 frontmatter、并被派生进 Profile 的 `schedules`，所以 `AgentScheduler` **一行不用改**——照旧遍历 `Profile.schedules` 注册 cron（25 节）。效果正是你要的：**一个 Agent 目录声明了 `schedules`，系统扫到就到点自动跑**，不用另配任何东西。

2.4 补运行时注册，去掉"重启"这条尾巴；L3 脚本的沙箱与信任边界

- **运行时注册（为 30 节铺路）**：`ProfileRegistry`（16 节）现在不可变；本节改成可变并发 Map，新增 `register(Profile)` / `remove(String)` / `exists(String)`，让"启动扫描"和"运行时新增 Agent"走同一段代码、同一套校验（同一异常、同一消息）。`AgentScheduler` 把 `registerAll` 循环体抽成 `registerProfile(Profile)`，新增一张 `Map<String, ScheduledFuture<?>>` 句柄表（30 节注销/更新用），与既有 `taskLocks` 并存。
- **L3 脚本的沙箱**：Agent 的脚本经底座 `shell` / `python` 跑，安全边界落在这里——白名单放行"解释器（`python` / `bash`）+ 限定只能跑这个 Agent 自己 `scripts/` 目录下的脚本"。
- **一个必须说清的信任边界**：脚本是**任意代码**，`python scripts/foo.py` 一旦放行，它能读写文件、**能自己发网络请求**——绕过 `http_get` 那道域名白名单（白名单只管内置 `http_get`，管不到子进程的网络）。所以结论要摆明：**装一个带脚本的 Agent = 信任写它的人**。核心阶段沙箱对脚本只做"解释器 + 目录"两道白名单；把第三方 Agent 关进受限容器 / 网络隔离是扩展阶段的事。做 Agent OS 要对这条诚实。

有几样先别做：Agent 版本管理、Agent 市场 / 共享、跨 Agent 的能力复用（本版每个 Agent 独立自足）、L3 脚本的容器 / 网络隔离（核心阶段只到解释器 + 目录白名单）、Agent 同名冲突策略、文件监听热加载（30 节做）——都放扩展阶段。这一节做到"一个目录派生成一个会自己跑的 Agent + 运行时注册就位"就够。

本节交付物（Spec-Kit 拆解锚点）：

- **代码**：`AgentLoader`（解析 `.oryxos/agents/<name>/` 目录 → frontmatter/正文/资源路径）；`deriveProfile(agentDir) → Profile`；启动扫描 `.oryxos/agents/` 并注册（装配层）；`ProfileRegistry.register/remove/exists`（16 节改造点，改可变 Map）；`AgentScheduler.registerProfile` + `scheduledTasks` 句柄表；`ContextLoader` 注入 Agent 正文（从目录主文件读，去 frontmatter）。

- **测试（harness）**：`AgentLoaderTest`、`DeriveProfileTest`、`AgentScanRegisterTest`、`ProfileRegistryRuntimeTest`、`AgentSchedulerRegisterTest`、`ProgressiveDisclosureTest`（正文进 prompt、脚本/参考按需经 read_file/shell）。
- **文件**：本节示例 Agent 目录 `daily-reconcile/`（§1.3–1.4 全文给出：`AGENT.md` + `scripts/reconcile.py` + `skills/report-format.md` + `REFERENCE.md`），四部分俱全，作手动路径的参照物 —— 由 spec-kit 按课件产出，不手工搓。
- **校验**：`AGENT.md` 缺 `name` / `provider` 等必填 → 加载报错点名；`tools` 里引用底座未注册的能力 → 加载告警。

三、怎么验收：把机制固化成 harness

这一节的机制是"扫描 → 派生 Profile → 注册 → 到点自己跑"，harness 把这条链每一环钉死，尤其钉住 **"目录派生的 Agent 与手写 Profile 走同一套校验/同一个底座"**：

测试类	覆盖的验收点
`AgentLoaderTest`	正确拆出 `AGENT.md` 的 frontmatter 与正文、认出 `scripts/` / `skills/` / `REFERENCE.md`；缺 `name` / `provider` 报错点名
`DeriveProfileTest`	frontmatter 各字段正确映射到 `Profile`；**`schedules` 原样带进派生的 Profile**（定时来自 Agent 的直接证据）
`AgentScanRegisterTest`	扫一个放了 N 个 Agent 目录的目录 → `ProfileRegistry` 出现 N 个 Agent、带 `schedules` 的都进了 `AgentScheduler`
`ProfileRegistryRuntimeTest`	`register()` 后立即 `get()` 可见；非法配置报错与启动路径**完全一致**（同一异常类型 + 同一消息）
`AgentSchedulerRegisterTest`	`registerProfile` 后 `scheduledTasks` 有句柄（30 节注销前提）；cron/时区来自 `Profile.schedules`
`ProgressiveDisclosureTest`	Agent 正文进 system prompt；参考/脚本**不预载**，靠底座 `read_file` / `shell` 按需取（渐进式披露守点）

"Agent 正文改了即时生效"不用在这节重测——17 节 `ContextLoaderTest` 的无缓存回归已经钉死。

验收清单（每条都成立才算达标）

- **一个目录定义一个 Agent**：往 `.oryxos/agents/` 放一个目录，`oryxos profile list` / `GET /api/v1/profiles` 里就出现这个 Agent，全程没写一行 Java；
- **定时来自 Agent**：Agent 的 `schedules` 到点真自己触发、webhook 收到、审计有账（真模型链路）；

- **资源按需加载**：Agent 跑到需要时才用 `read_file` 读参考 / 子指令、用 `shell` 跑脚本；脚本产出进上下文、代码不进；
- **正文即时生效**：改 Agent 目录里的正文不重启，下一次触发就用新说明；
- **两条来源同规矩**：目录派生的 Agent 与手写 Profile 过同一套校验（harness 钉死）；
- **运行时注册就位**：新扫入的 Agent 立即可见、定时留了句柄（为 30 节注销铺路）；
- **不回退**：`mvn clean verify` 全绿。

到这里，"在底座上定义一个会自己跑的 Agent"这件事，机制上完全成立了——只是入口还停在"登录服务器往目录里放文件"。下一节把最后一块拼上：把这套能力包成 `POST /api/v1/agents`，再加上"**一句话自动生成一个 Agent 目录**"，让业务系统 / 运营在页面上说一句话，就造出一个会自己跑的 Agent——管理平台也从"只能看"升级成"真能管"。